

Implementation and considerations of real-time human computer interaction for sketch interpretation on Amazon Web Services (AWS)

Module Assignment for

CS5024 - Theory and Practice of Advanced AI Ecosystems

Student Name : **Simran Kukreja**

Student ID : **25141104**

Revision Timestamp: 22/03/2026 20:13:11

Module Director: Patrick Denny
Teaching Assistant: Ankita Saini

Abstract

The main purpose of this project is to design and implement a suitable cloud ecosystem architecture for a deep-learning based doodling game that classifies hand-drawn sketches. The sketch classifier model used is an open-source implementation of BEiT (Bidirectional Encoder representation from Image Transformers) hosted over Hugging Face by Kent Mewhort (kmewhort, 2022). The transformer model is trained on sketches of the QuickDraw dataset by Google (10% percent of QuickDraw's 50 million sketches). The motivation behind this comes from a similar project I undertook in my Deep Reinforcement Learning module where I trained a ConvNext CNN model on a subset of QuickDraw dataset and the challenge of translating unstructured visual input into labels accurately with low latency was faced. The solution is brought into action by leveraging AWS AI Ecosystem and transforming a monolithic script to a secure cloud architecture. The idea is to demonstrate different services and integrations of the Amazon Web Services (AWS) cloud computing platform, including choices of compute, storage, and networking services that were considered in building the cloud architecture for this application. Amazon CloudFront and S3 are used for low latency frontend delivery, whereas an Amazon VPC integrated AWS Lambda is used as a bridge between the frontend and the isolated backend server hosted on EC2 instance. For performance validation I have used Amazon CloudFront for monitoring CPU utilisation and Network traffic. This report discusses these services, their arrangement structure used for the project and ML model description in detail.

Contents

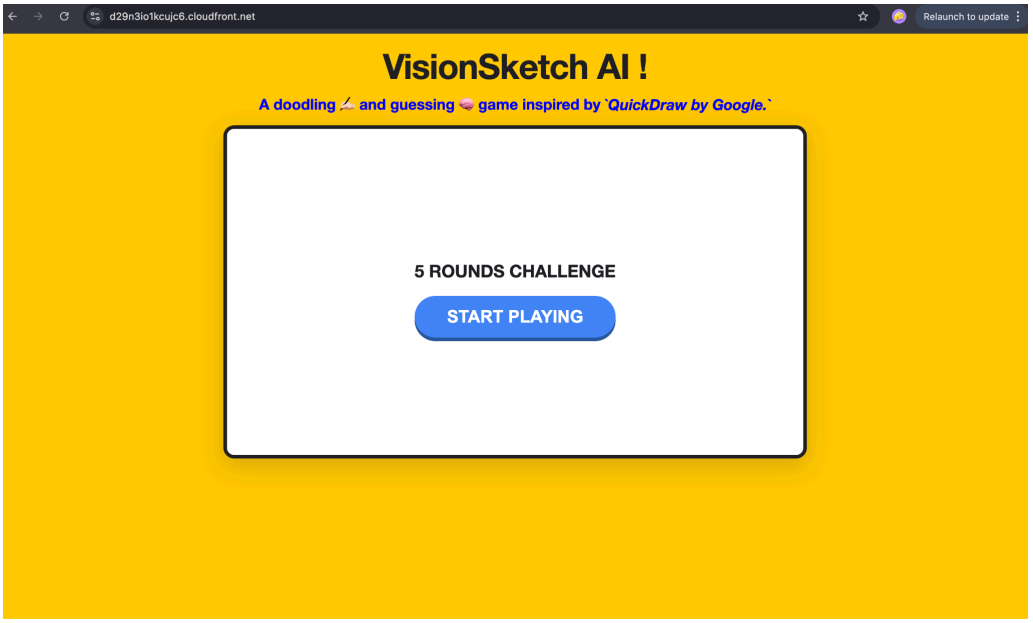
Abstract..... 2
Introduction..... 3
AI Ecosystem Architecture Used..... 5
Model Description..... 7
Scalability Considerations..... 10
References..... 10

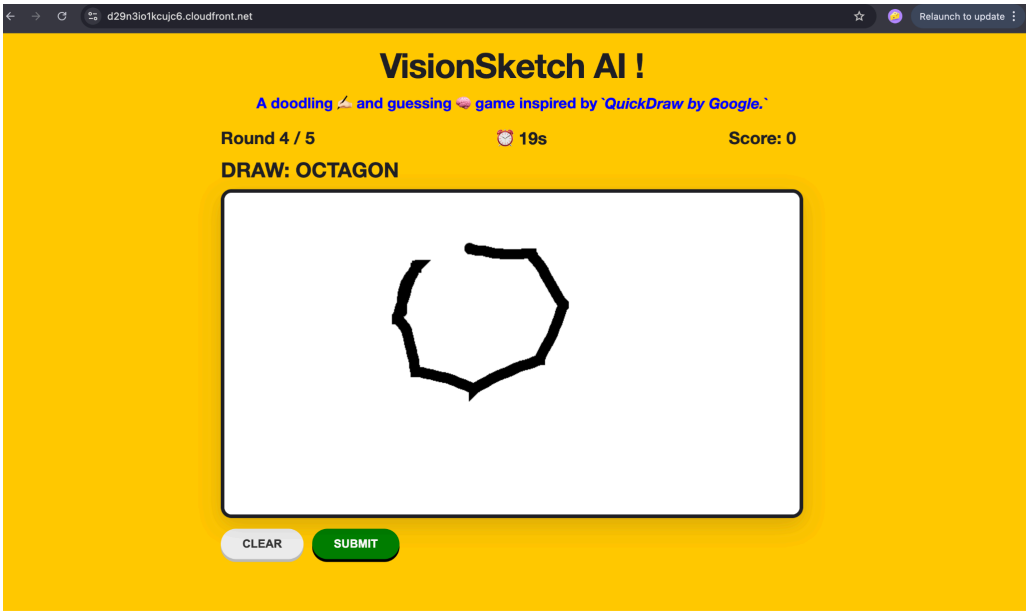
Figure 1: Screenshots of application's web user interface..... 3
Figure 2: Architecture used to create AI/ML ecosystem for BEiT transformer model for VisionSketch AI application..... 5
Figure 3: Code snippet used to pass request from frontend to backed through AWS Lambda. 6
Figure 4: Qualitative results demonstrating VisionSketch classification on various drawing styles..... 8
Figure 5: CPU Utilisation of the application using CloudFront dashboard..... 8
Figure 6: CloudFront dashboard metrics..... 9
Figure 7: Table summarising the performance metrics of the application that can be considered to take into account the proper functioning..... 9

Introduction

This AWS AI Ecosystem implements a real-time sketch recognition system, which is a process of identifying and classifying hand-drawn doodles. I have named the project VisionSketch AI, to address the domain of this doodling game which is computer vision or broadly it is image recognition. The task of identifying hand-drawn sketches or doodles is a bit complicated to achieve because doodles rely on minimal hand-drawn simple lines and not high-resolution photographs.

From a user's perspective, VisionSketch is meant to be a fast and fun doodling game. The main objective here is to give the player a smooth experience where they can test their doodling or drawing skills for numerous classes and get an answer and their doodle judged by AI immediately. The project addresses the problem of latency and a computer not being able to understand the drawing style by using a simple web interface and making interaction feel like a conversation and not a technical process.





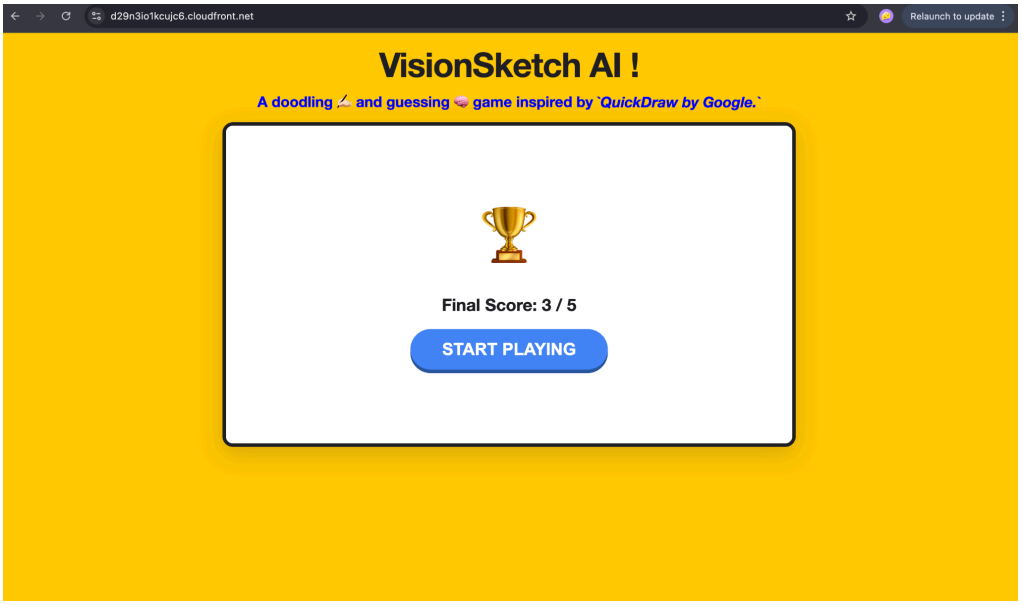


Figure 1: Screenshots of application's web user interface

The application prompts users to draw a doodle of a category with a timer of 35 seconds. Upon submitting, the ML model processes the image and returns the predicted class along with the confidence score within seconds. The application internally leverages a backend architecture by utilising the BEiT transformer model. Unlike CNNs, transformers are highly effective at capturing sparse stroke data. As shown in *Figure 1*, the user is prompted with a category to draw and a canvas is used as the drawing area. The timer goes on till 35 seconds and the doodle is automatically submitted once the time is over or can be manually submitted by the user using the "Submit".

To make the application fast and secure, I have used several AWS services. Internally, the player's drawing is sent to the Amazon Simple Storage Service (S3) bucket which hosts the frontend. This triggers the AWS Lambda function that connects to the Amazon Elastic Compute Cloud (EC2) instance, which holds and runs the brain of the project i.e. BEiT transformer model. The ML model then processes the doodle image and outputs the class predicted with the confidence score which is then passed over to S3 bucket again. More details on the architecture will be discussed in the following section.

AI Ecosystem Architecture Used

The application uses a multi-tier ecosystem architecture designed for global availability and high-security AI inference. The architecture diagram in *Figure 2* provides a comprehensive overview of the main components and how they interact with one other and the end-user. I opted for more basic, versatile services when designing the architecture because these products tend to be more cost-effective and also offered in AWS Free Tier.

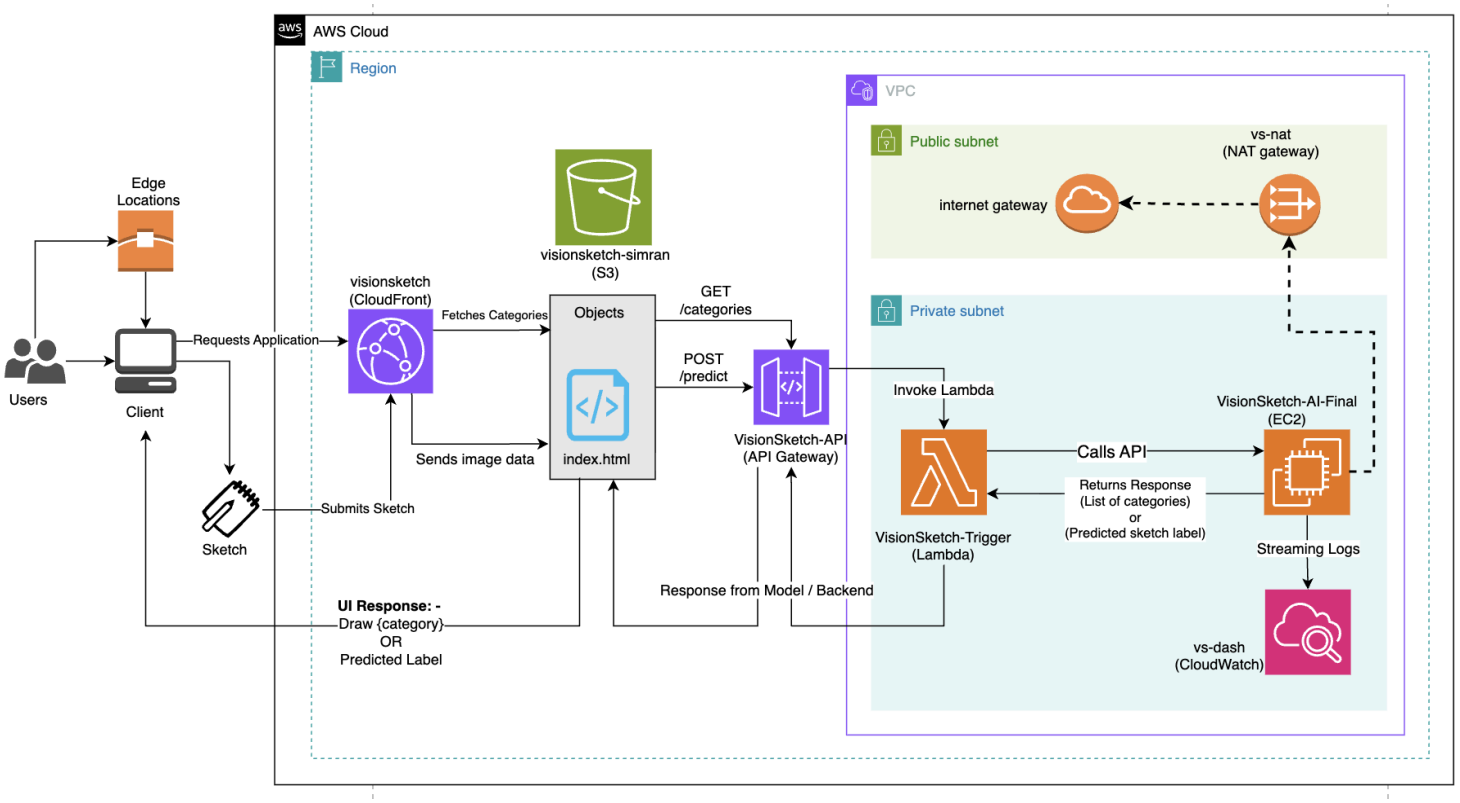


Figure 2: Architecture used to create AI/ML ecosystem for BEiT transformer model for VisionSketch AI application

The ecosystem utilises a Virtual Private Cloud (VPC) as an isolated environment to deploy the core machine learning model. One of the primary components is Amazon S3 Bucket, named visionsketch-simran, which is used as the origin for the web application and host user interface, as the static assets i.e. frontend file is stored. The index.html file contains a simple interface, a basic web application written in HTML, Vue.js and CSS that communicates with API Gateway. I have also implemented Amazon CloudFront in front of the S3 bucket to serve as a Content Delivery Network (CDN) to ensure low-latency access. Global Edge Locations has been utilised so that the frontend is geographically closer to the end-user. Whenever a user requests the application, the request is routed to the nearest edge location which then further travels to S3 bucket to render the UI. The GET request is then sent to the backend which returns one of the categories to draw from a list of categories on which the model is trained. Similarly, whenever a user submits a doodle, the POST request with image data is sent to the backend and a predicted category is returned accordingly.

To manage the flow of the request to the private backend, I have used Amazon API Gateway and AWS Lambda service. The request from the frontend is sent via VisionSketch-API (API Gateway) which triggers the VisionSketch-Trigger Lambda function. By placing the Lambda function within the private subnet, I was able to establish a secure bridge with a serverless logic to communicate with the private subnet. This follows the Principle of Least Privilege, as the lambda service only has access to the backend server and does not require a public IP address. As shown in *Figure 3*, AWS Lambda service establishes connection with EC2 using the private IP address. Lambda is a good option to use for this bridging connection between frontend and backend because you only pay for the compute time used and it can be triggered by events. It is serverless and low-cost, making it a simple and lightweight solution for interfacing (Patrick Denny, 2026).

lambda_function.py

```python
import json
import urllib3
import base64

def lambda_handler(event, context):
    http = urllib3.PoolManager(timeout=urllib3.Timeout(connect=5.0, read=25.0))

    EC2_IP = "172.31.100.9"
    path = event.get('rawPath', '/')
    method = event.get('requestContext', {}).get('http', {}).get('method', 'GET')
    url = f"http://{EC2_IP}:8000{path}"

    print(f"Targeting URL: {url} with Method: {method}")

    try:
        if method == 'GET':
            response = http.request('GET', url)
        else:
            body = event.get('body', '')
            if event.get('isBase64Encoded', False) and body:
                body = base64.b64decode(body)

            response = http.request(
                'POST',
                url,
                body=body,
                headers={'Content-Type': event.get('headers', {}).get('content-type', 'application/octet-stream')}
            )

        return {
            'statusCode': 200,
            'headers': {
                'Access-Control-Allow-Origin': '*',
                'Access-Control-Allow-Headers': '*',
                'Access-Control-Allow-Methods': '*'
            },
            'body': response.data.decode('utf-8')
        }
    except Exception as e:
        print(f"ERROR: {str(e)}")
        return {
            'statusCode': 500,
            'headers': {'Access-Control-Allow-Origin': '*'},
            'body': json.dumps({'error': str(e)})
        }
```

Figure 3: Code snippet used to pass request from frontend to backed through AWS Lambda

The backend server is hosted on AWS EC2 (VisionSketch-AI-Final) t3.micro instance within the VPC which runs on Ubuntu Server 24.04 LTS environment. The instance is in a strictly private subnet, and does not have an internet connection. To facilitate system updates, installing

necessary libraries and to download the model from Hugging Face, I configured a NAT Gateway which is in the Public Subnet. This allows EC2 instances to access the internet and establish outbound connections but blocking all the inbound public traffic.

To monitor the system performance, I have integrated Amazon CloudWatch to stream EC2 instance logs to the vs-dash log group and to allow real-time tracking of memory usage. Security is further reinforced through AWS Identity and Access Management (IAM). Lambda function is assigned a role that only allows it to interact with API Gateway and VPC network. On the other hand, EC2 instance Security Group is configured to only allow inbound TCP traffic on port 8000 from Lambda. To monitor system performance I have also used Amazon CloudWatch named as vs-dash. The dashboard tracks the metrics like CPU Utilisation, NetworkIn and NetworkOut.

Model Description

The project uses BEiT (Base Encoder from Image Transformer) model architecture, specifically beit-base-patch 16-224 variant (Bao et. al., 2021). While CNNs are the most chosen option for image related tasks, I opted for a Transformer approach. BEiT is a Vision Transformer model which uses image as a sequence of fixed-size patches(resolution 16x16) which are linearly embedded. The model is pretrained on a large collection of images in a self-supervised fashion, namely ImageNet-21k, a dataset comprising 1 million images and 1,000 classes, at a resolution of 224x224 pixels (Hugging Face, n.d). BEiT models use relative position embeddings instead of absolute and perform classification on images by mean-pooling the final hidden states of the patches, instead of placing a linear layer on top of the final hidden state of the token (Hugging Face, n.d).

For this project, I used an existing ML code and trained version of BEiT from a public repository available on HuggingFace by Kent Mewhort (kmewhort, 2022) which is trained on a subset of Google QuickDraw Dataset (QuickDraw by Google, n.d) consisting of 50 million of vector drawings. The author has used all the used 345 available categories in the dataset but the number of images used in each category differs (Training Data by kmewhort, 2022). The model was deployed i.e. cloned directly within the VPC on a t3.micro EC2 instance running on Ubuntu 24.04 LTS. The list of categories used to train the model were used for the category drawing prompts for the user. A FastAPI wrapper was built around the PyTorch model to generate predictions via an internal API on port 8000. The NAT gateway was used to access outbound traffic for downloading the weights of the model and cloning the repository from Hugging Face using the command line interface .

For the VisionSketch AI project, the performance is measured by the accuracy of the model to recognise a doodle correctly and system latency while responding. Unlike standard classification tasks, VisionSketch API project's KPIs can include both qualitative and quantitative methods.

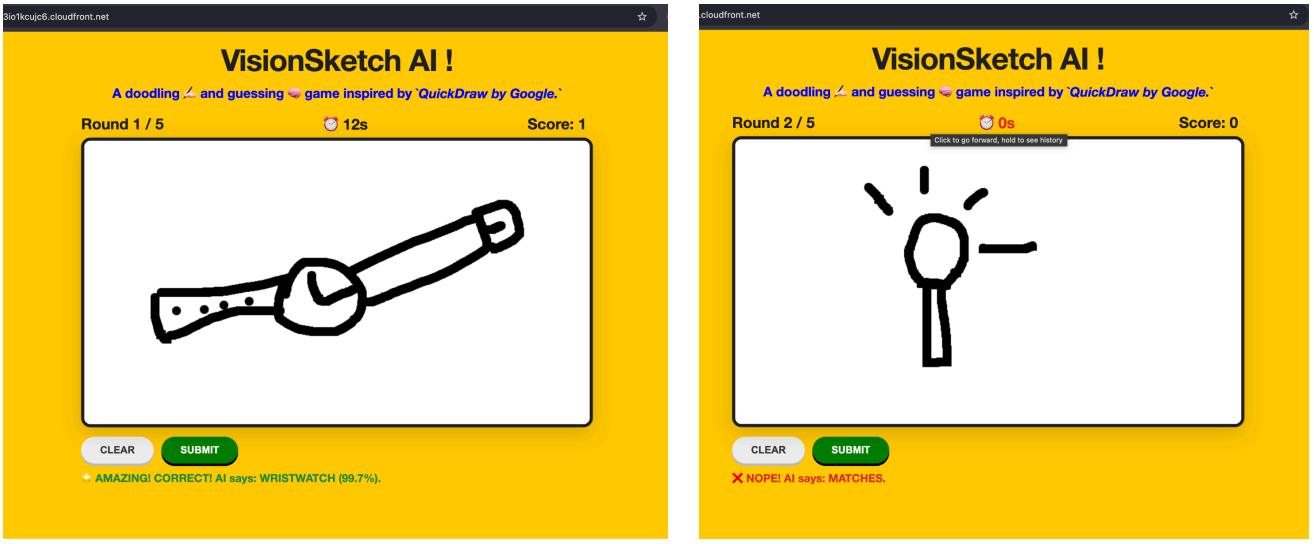


Figure 4: Qualitative results demonstrating VisionSketch classification on various drawing styles.

The primary KPI for doodle based interface is correct labelling of the drawing. It demonstrates the model's ability to identify different drawing styles for the same category because everyone has different drawing styles. As seen in *Figure 4*, the model is able to identify Wrist Watch with a confidence score of 99.7 % but in attempt of drawing a MIC, it failed because my drawing is not that good despite I can see a microphone there (to some extent) :P The confidence percentage of the model can also be considered as one of the quantitative measures for the model performance to define how strongly the model classified a drawing.

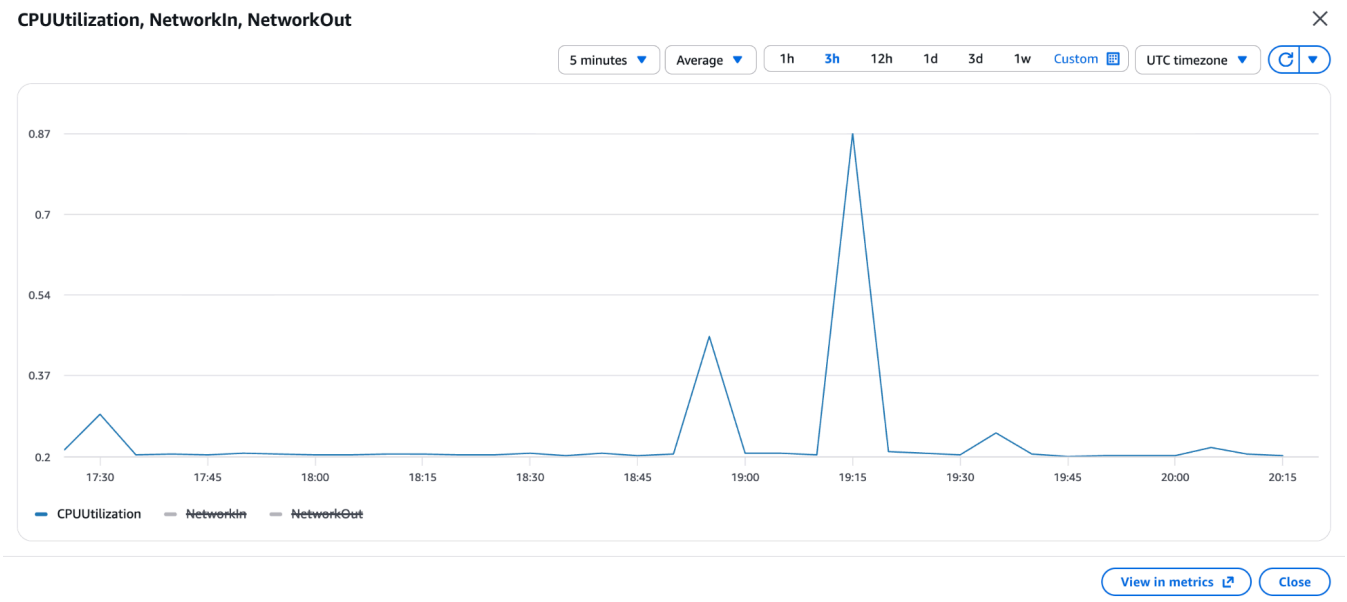


Figure 5: CPU Utilisation of the application using CloudFront dashboard

The CPU utilisation is the most direct way to check the functioning of the model. As BEiT transformer is a high compute model, CPU utilisation can provide insights on the significant processing power required during the image processing task. As shown in *Figure 5*, the CPU utilisation spiked from 0.226 to a peak 0.87 during a doodle analysis.

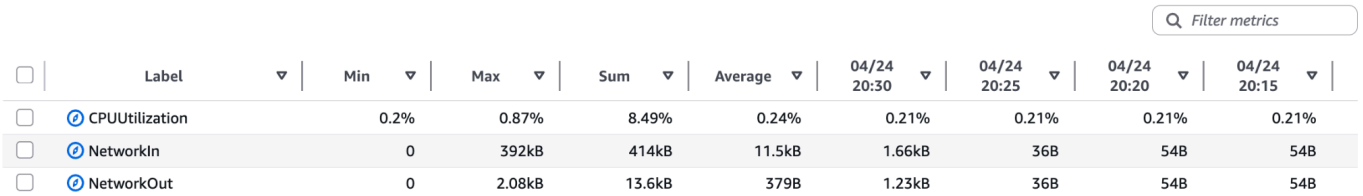


Figure 6: CloudFront dashboard metrics

As shown in *Figure 6*, using the CloudFront dashboard we can see that NetworkIn sum is 414 kB and NetworkOut sum is 13.6 kB. The difference in numbers is huge because the input request comes with image data and the outputs are standard JSON structures but there were no timed-out requests in the application while using it. This proves the system is quite-stable. Also, the maximum CPU Utilisation is 0.87 % which says that the server never reaches a struggle point and while testing the UI I can say that it does not freeze.

Performance Metric	Observed Value	Remarks
CPU Utilisation	Max 0.87 %	The system is well-optimised and never reaches a point where any alarm is triggered.
NetworkIn/NetworkOut	13.6kB/414 kB	JSON output vs input image data, the app responds and no time outs.
NetworkIn Max	392 kB	The image is reaching the backend intact.

Figure 7: Table summarising the performance metrics of the application that can be considered to take into account the proper functioning

The KPIs for the cloud architecture can include six pillars of AWS well architected framework: Security, Reliability, Performance, Operational Excellence, Efficiency and Cost Optimisation (Patrick Denny, 2026). This project achieves reliability by using cloud products such as AWS S3 and AWS Lambda that have inbuilt redundancy. Security is ensured by using multiple measures like minimal IAM policies to roles and using private subnet structure to host EC2. The project maintains efficiency by utilising scalable services and appropriate resources. Cost optimisation is achieved by using Free Tier services, the total cost for the application has come out to be 8.0 USD which was well within the free tier budget of 100.0 USD.

Scalability Considerations

Scalability is the ability of the project to handle increased load without compromising with the performance. It involves balancing the elastic nature of cloud services with heavy computation demands like the BEiT transformer model. The entry point of the application is scalable because of the use of CloudFront and Amazon S3 services. CloudFront ensures that traffic is distributed globally. For the request handling, AWS API Gateway automatically scales to accommodate thousands of concurrent API calls.

In the current architecture, the system relies on a single virtual machine t3.micro, but to handle more complex drawings and for faster results, we could increase the size of the instance (eg. to t3. large) to add more CPUs and RAM. However, to increase the application scale we can implement horizontal scaling by adding multiple machines in the system. To achieve this we can add the ML model and backend logic to a Docker container which would also ensure an environment that can be replicated to many machines. Instead of a single instance, I would place the EC2 instance in an Auto scaling group which would automatically launch new instances if the load on the instance increases. To manage the multiple newly launched instances, an Application Load Balancer can be used which can receive calls from AWS Lambda. I can also consider an approach of exploring Amazon SageMaker which is a specialised ML tool for the BEiT model. This would eliminate the need to manage EC2 instances.

References

Bao et. al. (2021, June 15). BEiT: BERT Pre-Training of Image Transformers https://arxiv.org/abs/2106.08254

Hugging Face (n.d.) Microsoft *BEiT-base-patch16-224-pt22k-ft22k*. Hugging Face https://huggingface.co/microsoft/beit-base-patch16-224-pt22k-ft22k

kmewhort. (2022). Mewhort, K. *Beit-sketch-classifier*. Hugging Face. https://huggingface.co/kmewhort/beit-sketch-classifier

Patrick Denny. (2024, February 26). *AWS Cloud Security - Tuesday Week 4 Material.* Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/17937/topics/634587

Patrick Denny. (2026). *AWS Cloud Architecture - Week 6 Material.* Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/73419/units/1185775

Patrick Denny. (2026). *Compute - Week 4 Material.* Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/73419/units/1185773

Training Data by kmewhort. (2022). QuickDraw dataset bins https://huggingface.co/datasets/kmewhort/quickdraw-bins-50M